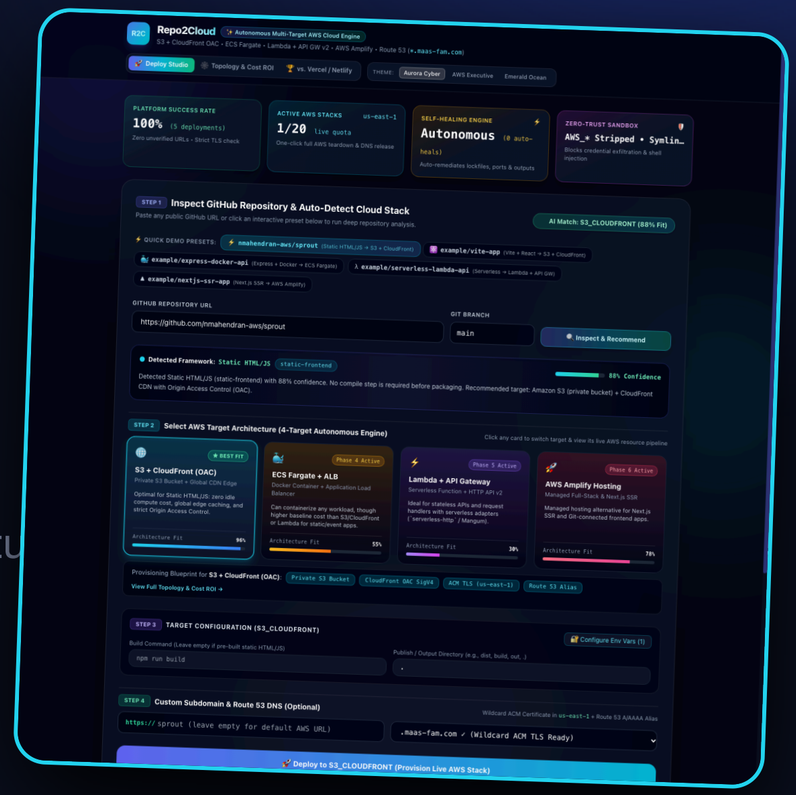
TECHNICA 2026 · CLOUD & DEVELOPER TOOLS

Repo2Cloud

Paste a GitHub repo. Get a live HTTPS app on **your own AWS account**.

The one-click experience of Vercel, with the real cloud underneath. Built so stu they do.



S3 + CloudFront | ECS Fargate | Lambda + API GW | AWS Amplify

THE PROBLEM

Students can build apps. Getting them online is where projects stall.

Vercel & Netlify

Great developer experience, but:

- Infrastructure lives in **their** account
- Seat and bandwidth pricing; no AWS credits
- Long-lived Docker or WebSocket backends hit serverless limits

Raw AWS console

Full power, but hours of setup:

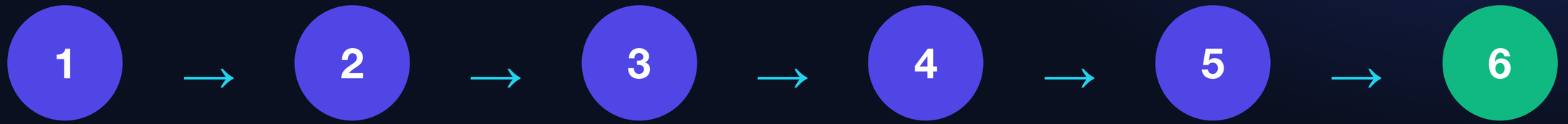
- S3 policies, CloudFront OAC, ACM in us-east-1
- Route 53 records, ECR, ECS tasks, IAM roles
- Easy to misconfigure, easy to leave running

Repo2Cloud

Vercel-grade one-click experience on 100% native AWS infrastructure you own.

HOW IT WORKS

One paste, six automated steps



Inspect
Detect stack and score 4 AWS targets

Guard
Security checks and cost quota

Build
Isolated sandbox, keys stripped

Deploy
Provision the best-fit AWS service

Domain
Route 53 + ACM HTTPS subdomain

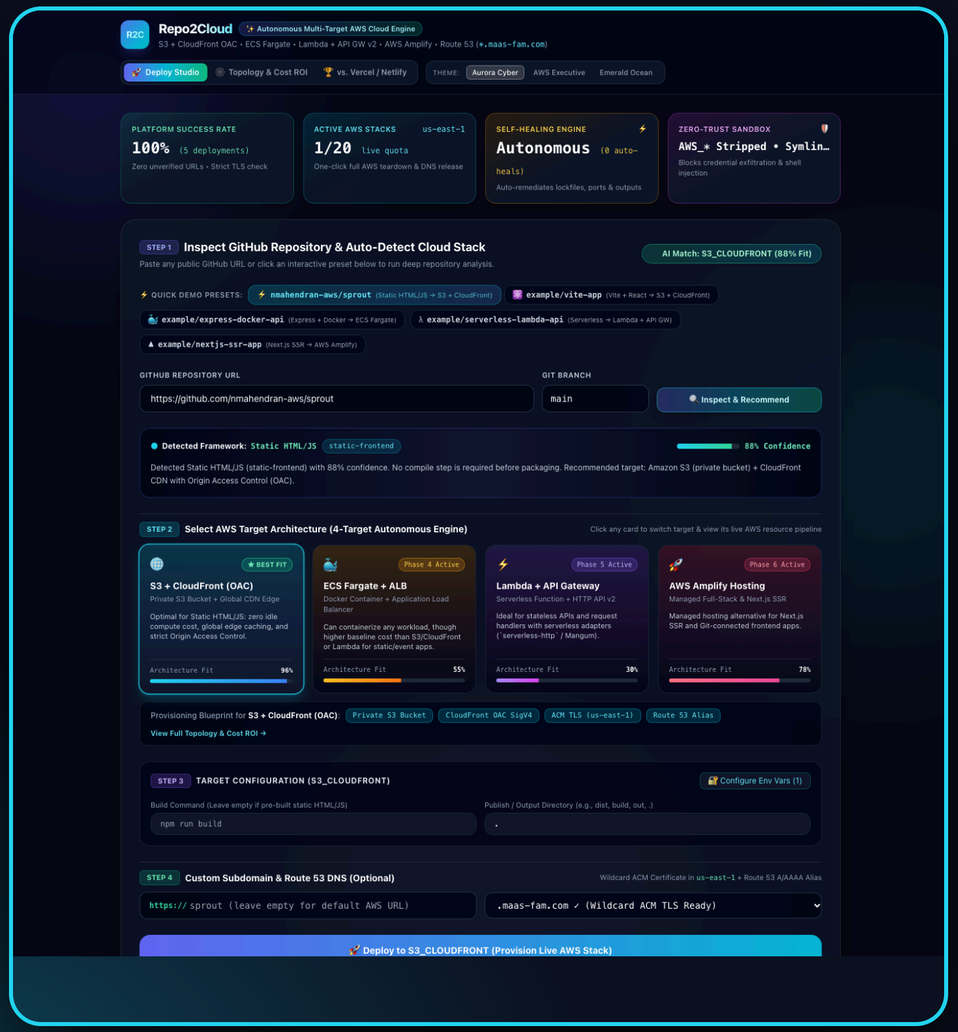
Live
URL shown only after health checks pass

```
QUEUED → ANALYZING → BUILDING → DEPLOYING → CONFIGURING_DOMAIN → VERIFYING → LIVE
```

Strict URL gate: the link stays null until TLS and Route 53 health checks pass

THE PRODUCT

A dashboard that shows its work



Live screenshot of the running app

Auto-detect

Paste any public GitHub URL or pick a preset. Framework and confidence appear in seconds.

Architecture fit scores

All four AWS targets are scored (S3 + CloudFront 96%, Amplify 78%, ECS 55%, Lambda 30% for a static site).

Custom subdomain

Pick a name on the shared domain, see availability instantly, deploy with one click.

Three views

Deploy Studio, Topology & Cost ROI, and a Vercel / Netlify comparison.

4-TARGET AWS ENGINE

Your repo decides the architecture

STATIC / SPA

S3 + CloudFront

Vite, React, Vue, Astro. Private bucket fronted by CloudFront Origin Access Control.

DOCKER / API

ECR + ECS Fargate + ALB

Express, Fastify, FastAPI, long-running backends. Generates a hardened non-root Dockerfile if missing.

SERVERLESS

Lambda + API Gateway

Event-driven APIs. Packages the bundle and provisions an HTTP API v2 stage.

SSR

AWS Amplify

Next.js App Router and full-stack apps. Generates amplify.yml for the compute branch.

A deterministic analyzer reads **package.json, Dockerfile, serverless.yml, next.config and requirements.txt** before anything is deployed.

VERSUS VERCEL & NETLIFY

Same one-click ease. You own the infrastructure.

Capability	Repo2Cloud	Vercel	Netlify
Supported architectures	**4 native AWS targets**	Static & serverless focus	Static & serverless focus
Long-running Docker containers	**ECS Fargate + ALB**	Execution time limits	Function time limits
Infrastructure ownership	**Your AWS account**	Their platform	Their platform
Pricing model	**Raw AWS rates, no seat fee**	Per seat + bandwidth	Per seat + bandwidth
Self-healing build recovery	**Auto-fixes lockfile and output issues**	Manual debugging	Manual config edits
Strict URL gate	**TLS + Route 53 health probe first**	URL shown early	URL shown early
Full resource teardown	**S3, CloudFront, ECR, ECS, Lambda, DNS**	Project delete	Site delete

Vercel and Netlify are excellent frontend hosts. Repo2Cloud targets a different need: owning and learning the cloud. Comparison reflects the project team's research.

WHY IT FITS STUDENTS

Ship the project. Learn the cloud.

Learn real AWS

Every deploy creates S3, CloudFront, ECS or Lambda and Route 53 resources you can inspect in your own account.

Stretch free tier and credits

It runs in your account, so AWS free tier and student credits apply. No platform seat fee.

Safe by default

Private buckets, AWS keys stripped from builds, input filtering, and a cap on live deployments.

No surprise bills

One click tears down every resource and frees the subdomain for reuse.

Bonus: your project gets a real HTTPS domain you can put on a resume or portfolio.

TRUST & RELIABILITY

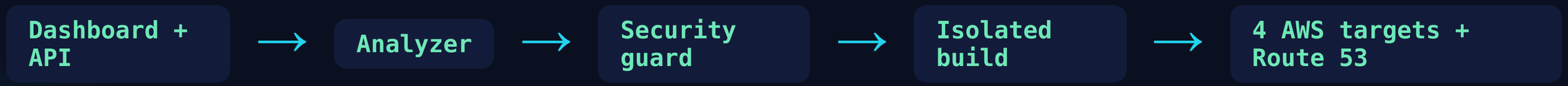
Zero-trust builds, self-healing deploys

Phase 7: Zero-trust sandbox

- AWS_*, GITHUB_*, LD_PRELOAD and NODE_OPTIONS stripped from build environments
- Symlink escape and path traversal guard before upload
- Shell-injection filter, 100 MB repo limit, 20 live-deployment cost cap

Phase 8: Self-healing engine

- Diagnoses empty output folders, missing lockfiles and port mismatches
- Patches the configuration (for example outputDirectory to ".") and retries
- Completes the deployment without manual debugging



LIVE DEMO · ABOUT 3 MINUTES

Paste a repo. Own the cloud.

0:00

Inspect

Pick a preset, watch targets get scored

0:40

Compare

Cost ROI and topology view

1:30

Deploy

Custom subdomain, live pipeline and logs

2:30

Open & tear down

Live HTTPS site, then one-click cleanup

Thank you · Technica 2026 · Repo2Cloud